FACEBLUR — Head Model Fine-Tuning Pipeline (complete walkthrough)

This is the full, end-to-end procedure for training your own head detector and shipping it as `head.pt` for FACEBLUR. It supersedes the shorter `TRAINING_GUIDE.md` and references the helper scripts:

- `sample_frames.py` — pull diverse frames from clips (now accepts a **folder** of clips)
- `bootstrap_labels.py` — pre-label them so you correct instead of draw
- `split_dataset.py` — arrange labels into the train/val layout + `data.yaml`
- `check_split.py` — **verify** the split is clip-level and per-domain (no leakage)
- `fix_split.py` — **repair** a leaked split by moving whole clips between train/val
- `diagnose_heads.py` — run a model over a clip/image at a chosen `imgsz` to catch train/inference mismatch
- `train_head.py` — optional two-phase (tune → long final run) training for a big time budget

Why fine-tune at all: no public head model is trained on helmeted, motion-blurred, side/back, or cut-off heads (CQB / body-cam), nor on dense catwalk-style crowds. The only reliable fix is training on frames from your own footage.

> **Important change from earlier versions.** FACEBLUR used to *union* `head.pt` with the person→head method, so head.pt "could only add coverage." That is no longer the default. When your own `head.pt` is present, the person→head geometry pass is now **disabled** (it was estimating head boxes from body boxes and firing on forward-held gear/weapons — e.g. weapon illuminators). So `head.pt` **now carries the hard cases on its own.** That makes the quality of your model matter more, and makes the rest of this document — especially honest evaluation — more important, not less. See Step 10 for the exact runtime behavior.

Effort reality check: the labeling is ~90% of the work; everything else is a command. Plan for 2–3 rounds of train → find misses → label misses → retrain.

Pipeline at a glance

```
clips ──sample_frames.py──▶ dataset_raw/ (frames)
      ──bootstrap_labels.py──▶ dataset_raw/ (frames + draft .txt)
      ──[correct in Yolo_Label]──▶ dataset_raw/ (frames + final .txt)
      ──split_dataset.py──▶ head_dataset/ (train/val + data.yaml)
      ──check_split.py──▶ verify clip-level split, no leakage   ◀── NEW
      ──fix_split.py──▶ repair if a clip leaked (optional)       ◀── NEW
      ──yolo detect train / train_head.py──▶ runs/.../best.pt
      ──evaluate per domain (held-aside clips)──▶ trust the number
      ──rename──▶ head.pt ──▶ next to FACEBLUR.exe (now also bundled by the installer)
```

What you'll produce: `best.pt` → rename to `head.pt` → place next to `FACEBLUR.exe`. A single-class (`head`) YOLO detector trained on frames that look like your real footage.

Step 0 — Decide scope

Goal	Labeled frames	Result
Proof-of-life (first run)	150–300	Catches easy heads; misses hard ones
Solid first model	500–800	Good on footage like what you labeled
Strong	1,500+	Robust across clips/lighting

Pull frames from several different clips, and **keep at least one whole clip aside per domain** that you never label — that untouched clip is your only honest test of real-world performance (Step 8). Diversity (lighting, distance, head pose, back-of-head, helmets, blur) matters more than raw frame count.

If you have more than one domain (e.g. helmet-cam AND catwalk) — read this

A model trained on one kind of footage will not transfer to a very different kind. Mixing both in the training set is correct (one `head.pt` covers both, and it prevents the model from forgetting the first domain when you add the second). But two traps:

- **Balance by head count, not frame count.** Crowded footage (catwalk, 8+ heads/frame) contributes far more *instances* per frame than sparse footage (helmet-cam, ≤3 heads/frame). A 50/50 *frame* split can be a 1:3 *instance* split, and the loss is driven by instances — so the dense domain quietly dominates and the sparse one weakens. Aim for rough balance in labeled heads, not labeled frames.
- **Keep a held-out test set for *each* domain**, and measure them separately (Step 8). A pooled score hides one domain underperforming behind another doing well.

Step 1 — Create the training environment

Training needs `torch` present, and a GPU build for any reasonable speed. Use a dedicated env so your delicate FACEBLUR build env stays untouched.

```
conda create -n yolotrain python=3.10 -y
conda activate yolotrain
REM GPU torch FIRST so ultralytics doesn't pull the CPU build:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
REM (use .../whl/cu118 if your Nvidia driver is older)
REM ultralytics brings cv2, numpy, pillow, pyyaml, matplotlib, etc.:
pip install ultralytics "numpy<2"
```

Verify:

```
python -c "import torch, ultralytics, cv2, numpy; print('torch', torch.__version__, '| CUDA', torch.cuda.is_available(), '| np', numpy.__version__)"
```

`CUDA True` = ready. `CUDA False` = it'll train on CPU (slow); try the `cu118` index instead. If the version string ends in `+cpu`, you have a CPU-only build and *must* reinstall from the CUDA index.

> **Sanity check before a long run:** run `nvidia-smi` to confirm the GPU is seen and to read your VRAM. On low-VRAM cards (e.g. GTX 1060 3GB) training can *silently spill into system RAM* instead of erroring — it won't OOM, it just runs slow. So "it didn't crash" is **not** proof it fit in VRAM. Watch the `GPU_mem` column on the first training epoch (Step 7).
>
> **YOLO version note:** this pipeline uses YOLO11 (`yolo11n/s`), which is stable and well-supported. Newer generations exist, but don't switch architectures mid-project — finish your YOLO11 model first; treat anything newer as a separate, measured experiment.

Step 2 — Sample frames to label

`sample_frames.py` now accepts a **folder** of clips, individual clip names, or globs — any mix.

```
conda activate yolotrain
REM a whole folder of clips:
python sample_frames.py footage_folder --out dataset_raw --per-clip 80
REM clips nested in subfolders (per-camera/date dirs):
python sample_frames.py footage_folder -r --out dataset_raw
REM or individual files / globs (still works):
python sample_frames.py clip1.mp4 clip2.mp4 --out dataset_raw --per-clip 80
```

Saves diverse, de-duplicated frames to `dataset_raw/` . Knobs:

Flag	Default	Meaning
`--per-clip N`	80	Aim for ~N frames per clip, spread across its length
`--every N`	off	Instead, keep 1 frame every N (overrides `--per-clip`)
`--diff D`	0.12	Skip frames less than D different from the last kept (0–1)
`--min-blur B`	0	Drop frames blurrier than B (0 = keep all, including blur — you want some)
`-r` , `--recursive`	off	When a folder is given, also search its subfolders

Recognized video types now include `.mp4 .avi .mov .mkv .webm .m4v .ts .mts .m2ts .wmv .flv .mpg .mpeg .3gp .ogv` (case-insensitive). If nothing is found, the script prints a diagnostic explaining why (wrong path, clips in subfolders → use `-r` , or an unrecognized extension).

Deliberately keep hard frames: motion blur, backs of heads, helmets, partial/cut-off heads, **and dense crowds**. Those are the high-value cases.

> **Tip:** `--per-clip` is *per clip*. Pointing at a folder of 20 clips with `--per-clip 80` yields up to ~1,600 raw frames before the diversity filter — a lot to hand-label. Start lower (`--per-clip 30-40`) for big folders and raise it for the clips that contain the hard cases.

Step 3 — Get Yolo_Label

Download the pre-built Windows binary from the **developer0hye/Yolo_Label** GitHub releases page and unzip it. No compiling. It's a local desktop tool — your footage never leaves your machine.

Step 4 — Bootstrap draft labels (so you correct, not draw)

Run a model you already have over the frames; it writes a YOLO `.txt` next to each image plus `classes.txt` , which Yolo_Label loads as existing boxes.

```
python bootstrap_labels.py dataset_raw
```

Flag	Default	Meaning
`--mode person`	person	COCO person → head region (best recall; boxes anyone with a torso)
`--mode head`	—	Run a head model directly (`--model head.pt`); tighter, misses hard cases
`--model FILE`	auto	Override the model file
`--conf C`	0.20	Confidence floor

`--mode person` is the better bootstrap because it catches more (including blurred/side/back heads of anyone with a visible body). Using person mode here for *drafting labels* is unrelated to the app's runtime behavior — it's fine even though the app now disables the person→head pass when a head.pt is present.

These drafts are a FIRST PASS, not truth. The person/head models systematically miss helmeted backs-of-heads and dense overlapping crowds — exactly what you're training to fix — so in Step 5 you must add every head they skipped. Trusting the drafts blindly bakes the same blind spot into your data.

Step 5 — Correct & complete labels in Yolo_Label

Open `dataset_raw/` in Yolo_Label. Define exactly one class: `head` (id `0`).

Labeling rules that make or break the model:

- Box the whole head: helmet/hair top down to chin/jaw, ear to ear.
- Label **every** head — front, side, back, helmeted, blurred, and partial/cut-off (box the visible part, right to the frame edge).
- In crowds, apply one consistent occlusion rule (e.g. label a head if ~half is visible, including heads partly behind another head) and apply it identically across every frame. Inconsistent occluded-head labels hurt crowd recall more than missing frames do.
- If you genuinely can't tell a blob is a head, skip it — consistency beats catching every pixel. Slightly generous boxes are fine (the app pads anyway); wildly loose boxes teach the model to fire on walls.
- Keep some head-free frames with empty labels — useful negatives.

Killing a specific false positive (e.g. weapon lights / gear)

If the model fires on a specific non-head object (a weapon illuminator, a reflector, a light), the fix is **targeted hard negatives**: include frames that contain that object with **every real head labeled and the object left unlabeled**. The unlabeled object becomes background the model learns to ignore. Use a meaningful batch (dozens of frames, including the object's brightest/"on" state), not a handful. Do **not** invent a "not-head" class — detection has no negative class; you simply don't annotate it. (Note: in the current app, the most common gear false positive — the person→head box landing on a muzzle — is already removed because the person→head pass is disabled when your head.pt is loaded; see Step 10.)

Saving: Yolo_Label writes `<image>.txt` automatically as you navigate between frames. There is no separate export step.

YOLO label format (what's in each `.txt`)

One line per box, all coordinates normalized 0–1:

```
0 x_center y_center width height
```

The leading `0` is the head class. An image with no heads has an **empty** `.txt`.

Step 6 — Split into train/val (and verify it)

```
python split_dataset.py dataset_raw
```

Creates:

```
head_dataset/
├── images/train , images/val
├── labels/train , labels/val
└── data.yaml   (absolute path, single class 'head')
```

Flag	Default	Meaning
`--out DIR`	head_dataset	Output dataset root
`--val F`	0.2	Validation fraction
`--seed N`	0	Random seed

It splits **by clip** (whole clips go to train or val together) so near-identical frames never leak across the split — the #1 cause of great metrics and bad real results. With only one clip it falls back to a random per-frame split and warns you.

`data.yaml` it writes:

```
path: C:/.../head_dataset
train: images/train
val: images/val
names:
  0: head
```

Verify the split — do not skip this (NEW)

Random-frame leakage is the single most common reason for "great val numbers, bad real footage." Confirm the split is honest:

```
python check_split.py --root head_dataset
```

It groups every image by clip and reports, per split, how many clips/frames there are — and flags any clip that appears in **more than one** split (that's leakage). If you have multiple domains, the clip list also lets you confirm each domain appears in val, not only in train.

If it reports a leaked clip, repair it by moving that whole clip to one side (dry-run first, then `--apply`):

```
python fix_split.py --root head_dataset --to-train LEAKED_CLIP           REM dry-run
python fix_split.py --root head_dataset --to-train LEAKED_CLIP --apply   REM do it
python check_split.py --root head_dataset                                REM confirm clean
```

`fix_split.py` moves each image **and its matching label** `.txt` together, so nothing is orphaned.

Step 7 — Train

```
conda activate yolotrain
yolo detect train model=yolo11s.pt data=head_dataset/data.yaml epochs=100 imgsz=960 batch=8 patience=30 name=head_v1
```

Key flags:

Flag	Why
`model=yolo11s.pt`	Start from COCO-pretrained **small**. Prefer `s` over nano now: since the person→head pass is disabled when your head.pt is loaded, **head.pt carries all the hard-head recall alone**, and `s` has more capacity for crowded/occluded heads. Nano is fine for a quick proof-of-life.
`imgsz=960`	Biggest lever for small/blurry heads. **Must match the app** — see the box below.
`batch=8`	On a 3GB card, `batch=8` at 960 spills into system RAM (slow, not an error). `batch=4` fits real VRAM and trains faster. Raise on a big GPU.
`epochs=100` + `patience=30`	Ceiling + early stop when val stops improving.

> ⚠ **imgsz MUST match the app's `HEAD_INFER_IMGSZ`**
>
> The app runs the head model at `HEAD_INFER_IMGSZ` (default **960**) in `face_blur.py`. **Train at the same size you run at.** A train/inference size mismatch makes the model misread objects at scales it never trained on — this is what caused the weapon-illuminator false positives (model trained at 960, app running at the wrong size). Rule: - Train at **960** → leave `HEAD_INFER_IMGSZ = 960`. (Recommended; no app change.) - Train at **1280** → you **must** set `HEAD_INFER_IMGSZ = 1280` in `face_blur.py`. This improves small/distant-head recall but makes the app's head pass slower — noticeably on the **CPU** machines many users run. Treat the jump to 1280 as a deliberate, separately-verified change, not a free upgrade, and re-test your false-positive clips with `diagnose_heads.py` afterward.

Default augmentation (mosaic, flips, HSV) already helps pose/blur variation — no need to tune it for a first model. `close_mosaic` (last ~10 epochs without mosaic) is on by default and good for crowded scenes. Output lands in `runs/detect/head_v1/`; the weights are `runs/detect/head_v1/weights/best.pt`.

Confirm GPU during the run, not after: the startup banner should read `CUDA:0 (Your GPU, NNNN MiB)`, not `CPU`. Watch the `GPU_mem` column on epoch 1 — if it exceeds your card's VRAM, you're spilling into system RAM (slow); lower `batch` to fit.

Out-of-memory? Lower `batch` first (8→4→2), then `imgsz` (960→640).

Optional: maximum-accuracy run for a large time budget (`train_head.py`)

If you can leave the machine training for a long stretch (e.g. a weekend) and want the most accurate model your card can produce, `train_head.py` runs a two-phase plan: a fast hyperparameter **search** at low resolution, then one **long final run** with the tuned settings, then a held-out evaluation.

```
python train_head.py --data head_dataset/data.yaml               REM full plan (defaults to 960, matches the app)
python train_head.py --data head_dataset/data.yaml --no-tune     REM skip the search
```

It defaults to `yolo11s`, `imgsz=960`, `cos_lr`, and a long `epochs/patience`. It deliberately keeps `imgsz` matched to the app default. If you raise `--final-imgsz 1280`, remember the `HEAD_INFER_IMGSZ` rule above. On a 3GB card it accepts the RAM spill (slow but fine if time is free); check the epoch-1 ETA and lower `--epochs` if it won't fit your window. `multi_scale` training (trains across a range of input sizes) is worth considering specifically because it reduces the scale-sensitivity that caused the illuminator issue — but it raises peak memory, so use a lower base size on 3GB.

Step 8 — Evaluate honestly (per domain)

Look at the metrics ultralytics prints in `runs/detect/head_v1/` — especially **recall** (you care about not missing heads more than the odd false box; the app pads and you can over-cover for privacy).

Then test on the clip(s) you held aside and never labeled — the only honest measure:

```
copy runs\detect\head_v1\weights\best.pt head.pt
python test_head.py path\to\held_aside_clip.mp4
```

Open the saved `_headNN.jpg` frames and look specifically at backs-of-heads, blurred heads, and dense overlaps. **Counts alone lie — look at the pictures.**

If you have multiple domains, measure them separately. Run the held-aside helmet-cam clip and the held-aside catwalk clip independently and compare recall on each. A pooled number can show a healthy average while one domain regressed — and after rebalancing toward a dense domain, the *sparse* domain (helmet-cam) is the one most likely to have softened. You can also quickly test the model at the exact size the app uses (and at others, to detect a scale mismatch):

```
python diagnose_heads.py held_aside_clip.mp4 --model head.pt --imgsz 960
python diagnose_heads.py suspect_frame.jpg  --model head.pt --imgsz 960 1920   REM compare sizes
```

If a false positive appears at one size but not at 960, that's a train/inference mismatch, not a data problem — fix it by matching `HEAD_INFER_IMGSZ`, not by retraining.

Step 9 — Iterate (where the quality comes from)

Wherever it still misses on the held-aside clip:

1. `python sample_frames.py held_aside_clip.mp4 --out round2_raw --per-clip 80`
2. Bootstrap + correct those frames (focus on the failure type, e.g. backs of heads, dense crowds).
3. Copy the new frames+labels into `head_dataset/images/train` and `head_dataset/labels/train`, re-run `check_split.py`, then retrain as `name=head_v2`.

For **false positives** (firing on gear/walls), add **hard negatives** (Step 5) rather than more missed-head frames. For **scale-dependent** false positives, check `imgsz` matching with `diagnose_heads.py` before retraining.

Two or three rounds of this beats one giant first batch. It's normal — plan for it.

Step 10 — Ship it

`runs\detect\head_vN\weights\best.pt` → rename to `head.pt`, place next to `FACEBLUR.exe`.

Shipping to all users: `head.pt` is now bundled by the build. `installer.iss` installs it next to the exe:

```
Source: "head.pt"; DestDir: "{app}"; Flags: ignoreversion
```

and the build scripts copy it alongside the exe (`build.bat` copies it into `dist\`; `build_installer.bat` checks it's present before building). Just drop your latest `head.pt` in the project root before building.

How the app uses it (current behavior)

With **Detect whole head** on, FACEBLUR auto-detects the head class from the model's own metadata and runs `head.pt` at `HEAD_INFER_IMGSZ` with edge strips. The union logic has changed:

- **When your `head.pt` is loaded:** the **person→head geometry pass is disabled** (`PERSON_HEAD_MODE = "user_off"` in `face_blur.py`). Detection = `head.pt` ∪ face model. This removes the geometry false positives where the estimated head region landed on forward-held gear/weapons — but it means `head.pt` carries all the hard-case recall, so it must be good (hence the emphasis on evaluation above).
- **When no user `head.pt` is present** (only the auto-downloaded default, or none): the person→head pass still runs as the robust fallback.

A face already covered by a head box is not censored twice. Turn on **Show debug boxes** to see it work: **red** = `head.pt` firing, **yellow** = person→head region (won't appear when your head.pt is loaded), **cyan** = the face model.

`PERSON_HEAD_MODE` is a tunable constant if you want a different policy: `"user_off"` (default), `"any_off"` (disable whenever any head model loads, including the default), `"always"` (legacy union), `"never"` (fully off).

Full command cheat-sheet

```
REM one-time env
conda create -n yolotrain python=3.10 -y
conda activate yolotrain
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
pip install ultralytics "numpy<2"

REM per round
python sample_frames.py footage_folder -r --out dataset_raw --per-clip 80
python bootstrap_labels.py dataset_raw
REM ... correct + add missed heads in Yolo_Label (one class: head) ...
python split_dataset.py dataset_raw
python check_split.py --root head_dataset                REM verify split (NEW)
REM ... if a clip leaked: fix_split.py --to-train CLIP --apply, then re-check ...
yolo detect train model=yolo11s.pt data=head_dataset/data.yaml epochs=100 imgsz=960 batch=8 patience=30 name=head_v1
REM ... or: python train_head.py --data head_dataset/data.yaml  (long, max-accuracy)
copy runs\detect\head_v1\weights\best.pt head.pt
python test_head.py held_aside_clip.mp4                  REM per-domain, look at pictures
REM ... iterate on misses, then drop head.pt in the project root and rebuild ...
```

Troubleshooting & pitfalls

Symptom	Cause / Fix
Great val metrics, bad real footage	Train/val leakage. Run `check_split.py`; repair with `fix_split.py`. Split by clip, and never label your held-aside test clip.
One domain good, another bad in real life	Pooled metrics hiding a per-domain weakness, or instance imbalance toward the dense domain. Measure each domain separately (Step 8); balance by head count.
Model fires on a weapon light / gear at high conf	First check **imgsz matching** with `diagnose_heads.py` (train/inference mismatch). If it's the yellow person→head box, it's already disabled when your head.pt is loaded. If it's the red head.pt box, add hard negatives (Step 5).
Model fires on walls/gear generally	Loose/inconsistent boxes, or too few negatives. Tighten labels; keep some head-free frames.
Misses small/distant heads	`imgsz` too low. Raise to 1280 — **and** set `HEAD_INFER_IMGSZ=1280` to match.
Misses backs-of-heads / crowded overlaps	You didn't label enough of them. Over-represent the failure type; apply a consistent occlusion rule.
Out-of-memory in training	Lower `batch` (8→4→2), then `imgsz` (960→640).
Training crawls but never OOMs (low-VRAM card)	`batch` is spilling into system RAM. Lower `batch` to fit real VRAM; check the `GPU_mem` column and `nvidia-smi`.
`CUDA False`, training crawls	CPU torch build. Reinstall from the CUDA index (Step 1); try `cu118` if `cu121` mismatches your driver. A `+cpu` version string means CPU-only.
`split_dataset.py` says "no .txt"	Labels weren't saved. Yolo_Label writes `<image>.txt` as you navigate; make sure you moved through the frames.
App still shows face-only with `head.pt` present	Confirm `head.pt` is next to the exe and **Detect whole head** is on; check the log line naming the head method.

Appendix — Training scripts reference (`train_all.py` / `train_head.py`)

This expands on Step 7. It covers the two training helper scripts in detail: which to use, their flags, and worked examples.

What each script is for

Script	Trains	Use it when
`train_all.py`	Several sizes (nano/small/medium) back-to-back	You want to **compare sizes** or ship a size selector. One command, unattended, produces `head_n.pt` / `head_s.pt` / `head_m.pt` + a comparison.
`train_head.py`	**One** model, with an optional hyperparameter search first	You want the **best single model**, and you're willing to spend time on an automated tune before a long final run.

Both are for the same goal — a `head.pt` for FACEBLUR — just different strategies. For most FACEBLUR work, start with `train_all.py`: it's simpler, and it tells you which size actually performs best on your data (which, per our testing, is often **not** the biggest one).

Before you run either (prerequisites)

1. **Activate the training env:** `conda activate yolotrain`
2. **Confirm the GPU is really used** (a mismatch silently falls back to CPU and is ~100x slower): `python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"` You want `True` and your GPU name (e.g. `NVIDIA GeForce RTX 3060`).
3. **Confirm the dataset split is clean** (clip-level, no leakage): `python check_split.py --root head_dataset`
4. **For unattended runs, disable Windows sleep:** `powercfg /change standby-timeout-ac 0`

`train_all.py` — train nano/small/medium in one go

```
python train_all.py --data head_dataset/data.yaml
```

That trains `yolo11n`, `yolo11s`, `yolo11m` in sequence at imgsz 960, copies each to `head_n.pt` / `head_s.pt` / `head_m.pt`, evaluates each, and writes `training_summary.txt` comparing them.

Flags

Flag	Default	Meaning
`--data`	*(required)*	Path to `data.yaml`
`--models`	`n s m`	Which sizes to train (from `n s m l x`)
`--imgsz`	`960`	Training size — **must match the app's** `HEAD_INFER_IMGSZ`
`--epochs`	`300`	Epoch ceiling per model
`--patience`	`60`	Early-stop after this many epochs with no val gain
`--batch`	`-1`	`-1` = auto-size to VRAM per model (recommended)
`--device`	`0`	CUDA index, or `cpu`
`--out`	`.`	Where to copy the renamed `head_<size>.pt`

Examples

```
REM just nano and small (skip the heavy medium)
python train_all.py --data head_dataset/data.yaml --models n s

REM shorter runs that stop nearer the peak (see "Tips" — models overfit late)
python train_all.py --data head_dataset/data.yaml --models n s --epochs 80 --patience 25
```

What you get

- `head_n.pt`, `head_s.pt`, `head_m.pt` in `--out`
- `training_summary.txt` — a recall/mAP comparison table
- Per-model runs under `runs/detect/head_<size>/` (curves in `results.png`, weights in `weights/best.pt`)

Robust for unattended runs: if one size errors out, it logs the failure and continues to the next, so you still get the others.

`train_head.py` — one model, with an automated tune

```
python train_head.py --data head_dataset/data.yaml --final-imgsz 960
```

Runs in two phases: 1. **Search** — ~15 short trials at low resolution to find good augmentation / learning-rate values (fast). 2. **Final** — one long run at full resolution using those values, then a test-split evaluation.

Flags

Flag	Default	Meaning
`--data`	*(required)*	Path to `data.yaml`
`--model`	`yolo11s.pt`	Base weights to start from
`--name`	`head_final`	Run name
`--no-tune`	off	Skip the search phase (go straight to the final run)
`--tune-imgsz`	`640`	Resolution during the search (small = fast)
`--tune-batch`	`16`	Batch during the search
`--tune-epochs`	`20`	Epochs per search trial
`--iterations`	`15`	Number of search trials
`--final-imgsz`	`1280`	Final-run resolution — **see the warning below**
`--final-batch`	`8`	Final-run batch
`--epochs`	`300`	Final-run epoch ceiling
`--patience`	`60`	Early-stop patience
`--device`	`0`	CUDA index, or `cpu`
`--no-test`	off	Skip the held-out test evaluation

> ### ⚠ Override `--final-imgsz` to 960 for FACEBLUR
>
> `train_head.py` defaults `--final-imgsz` to **1280**, but the FACEBLUR app runs the head model at `HEAD_INFER_IMGSZ = 960`. **Train and run at the same size**, or you get scale-mismatch false positives (the weapon-illuminator problem). So for FACEBLUR, pass `--final-imgsz 960` — unless you also change `HEAD_INFER_IMGSZ` to 1280 in `face_blur.py` and accept slower CPU inference.

Examples

```
REM full plan, matched to the app
python train_head.py --data head_dataset/data.yaml --final-imgsz 960

REM skip the search, just one long run
python train_head.py --data head_dataset/data.yaml --final-imgsz 960 --no-tune

REM train a nano model instead of the default small
python train_head.py --data head_dataset/data.yaml --model yolo11n.pt --final-imgsz 960
```

What you get

- `runs/detect/head_final/weights/best.pt` — rename to `head.pt` to ship
- Printed recall / mAP on the test split

Which should I use?

- **Deciding which size to ship, or shipping the size selector →** `train_all.py`.
- **You already know the size and want the single strongest model →** `train_head.py` (start from `yolo11n.pt` or `yolo11s.pt`, `--final-imgsz 960`).
- **Quick iteration between labeling rounds →** `train_all.py --models n s --epochs 80` (fast, and nano/small are what perform best here anyway).

After training — deploy the model

1. Pick the winner from `training_summary.txt` (or `head_final/best.pt`).
2. **Confirm it's `best.pt`, not `last.pt`** — `best.pt` is saved at the peak epoch; `last.pt` is the (often overfit) final epoch.
3. Rename to `head_n.pt` / `head_s.pt` / `head_m.pt` (or `head.pt`) and place in the project root, then rebuild. The installer bundles them next to the exe.
4. Set `HEAD_INFER_IMGSZ` in `face_blur.py` to the size you trained at (960).

Tips (from testing on this dataset)

- **Bigger is not better here.** On our data, nano beat small beat medium on the hard (catwalk) domain — the larger models **overfit** the limited dataset. Check each run's `results.png`: if `val/box_loss` turns upward while `train` keeps dropping, that's overfitting. Prefer the smaller model unless you've added a lot more data.
- **Models peak early, then drift.** Val metrics often peak well before the epoch ceiling. `best.pt` captures the peak, but you can also stop sooner (`--patience 25`, `--epochs 60-80`) to save time.
- **The bottleneck is data, not model size or epochs.** If recall is low on a domain, add more (hard) labeled frames for that domain — you can't out-train a data limitation.
- **Always confirm the GPU banner** at the start of a run: `CUDA:0 (Your GPU …)`, not `CPU`. Watch the `GPU_mem` column stays under your VRAM (else it's spilling to system RAM and running slow).
- **Evaluate per domain** afterward with `eval_domains.py` — a pooled score hides one domain underperforming behind another.

FACEBLUR head fine-tuning pipeline — for use with `sample_frames.py`, `bootstrap_labels.py`, `split_dataset.py`, `check_split.py`, `fix_split.py`, `diagnose_heads.py`, `train_head.py`.